Doc. no:	P0782R0
Date:	2017-09-25
Reply to:	Erich Keane
	Adam David Alan Martin
	Allan Deutsch

A Case for Simplifying/Improving Natural Syntax Concepts

Introduction

Toronto's ISO C++ Meeting contained one of the most bittersweet moments in recent C++ Committee straw polls: The EWG vote to include the Concepts TS into the Working Draft without the Natural (aka "Terse" aka "Cute" aka "Abbreviated") Syntax. The discussion resulted in some extremely powerful tools that many in the C++ community appreciate incredibly, but places a component that many believe as vital to the success of the feature in limbo.

This Syntax aims to simplify the usage of Concept functions in order to make Generic Programming accessible by all levels of C++ programmers. However, one of the criticisms of this syntax is that it simplifies the syntax without simplifying the semantics The fear of many committee members is that the simplified syntax will hide the template-like semantics behind a behavior which hides in the costume of a normal function.

This proposal aims to alter a major component of the semantics of the "Natural Syntax" in a way that the authors of this paper believe with make the semantics match the Naturalness of the syntax.

Motivation

The authors of this paper believe that lowering the barriers to Generic Programming in C++ is an admirable goal. However, they also believe that the barrier to entry is NOT the syntax itself, but the semantics of C++ templates themselves. Paramount to these semantic differences is the way in which functions are looked up. Two phase template lookup often results in behavior that is unsurprising to experts, yet a minefield for anyone without extensive experience in writing templates. To illustrate, consider the following example:

```
template< typename Thing >
concept bool ConstIndexable = requires( Thing t ){ const_cast< const Thing&<( t )[ 0 ]; };
class SafeMap { public: void operator[] ( int ) const {std::cerr<< "SafeMap Const Indexer\n";} };
class MutableMap { public: void operator[] ( int ) {} };
class AdapiveMap {
  public:
    void operator[] ( int ) const { std::cerr << "Const Indexer\n"; }
    void operator[] ( int )       { std::cerr << "Non-Const Indexer\n"; }
};
void friendly( ConstIndexable &c ) { c[ 0 ]; }
int main() {
  { SafeMap safe; friendly( safe ); }
//  { MutableMap mut; friendly( mut ); } // doesn't satisfy ConstIndexable.
  { AdapiveMap adaptive; friendly( adaptive ); }
}
```

In this example, the user specifies that function `friendly` should take a type that is ConstIndexable. In the case of `SafeMap`, this works as expected, and in fact the attempt to call this funcction with `MutableMap` doesn't compile, since it doesn't satisfy the concept. However, the call with `Adaptive Map` will actually call the NON const version.

This behavior is quite surprising to many. A poll run by the authors of more than 130 university students resulted in nearly 3/4 of the students (and 2/3 of this paper's authors) believing that `AdaptiveMap`'s `const` index operator would be called by `friendly`.

The authors of this paper used this information (as well as some additional straw polls) to conclude that one of the issues with the Terse Syntax is that it hides some of the nastier properties of templates by making it non-obvious that this is a function template. Additionally, this behavior is a surprise to many programmers anyway, reducing templates to an 'expert' feature. It seems contrarian that a feture intended for beginners (Concept Terse Syntax) would immediately expose the programmer to an expert-level feature. This paper proposes a modification to a Terse Syntax Function's lookup rules that will make them less error prone and more intuitive for beginners.

Suggested Solution

This paper proposes a different mechanism for resolving dependent calls in a Concept Terse Syntax Function. First, a quick conceptual review of how a concepted function template has its dependent calls resolved:

1. The function template is parsed, and checked, and non-dependent calls and types are resolved.
2. Upon an attempt at instantiation, the substituted type is checked against the Concept specified.
3. All dependent calls and types are looked up using traditional lookup rules.

It seems unfortunate that all of the work done in step 2 is immediately discarded. This information is incredibly useful, and contains information that describes the intent of the template author and template invoker. If this information is kept for the 3rd step above, we can ensure that the programmer's intent is properly reflected in the generated code. This paper proposes that these 3 conceptual steps be replaced with the following for Concept Terse Syntax functions:

1. The function template is parsed, and checked, and non-dependent calls and types are resolved.
2. Upon an attempt at instantiation, the substituted type is checked against the Concept specified. **Any function/type that is used to satisfy the Concept or requires-clause is placed into a conceptual temporary, invisible namespace.**
3. All dependent calls and types are looked up**, however can only be taken from the namespace generated in step 2. No other function/type will be considered. Attempting to use a function/type not specified in the Concept is ill-formed.**

The authors believe that this functionality has numerous ohter advantages, however its most powerful feature is the simplification of one of the more difficult rules in the language, resulting in a more beginner-friendly feature.

Conclusion

The authors of this paper respectfully request feedback on the direction of this paper. They believe that this change will make the Terse Syntax more palatable for the committee as well as less surprising to developers. Note that this paper does not propose any changes to normal function-templates, which is still extremely useful for both existing code as well as an escape hatch for developers who require this behavior.